

NODE INTRODUCTION ASSIGNMENT

Theory assignment:

Introduction to node.js

Node is an open source, cross-platform javascript runtime environment that allows developers to run javascript code outside the browser. It is mainly used for building fast, scalable, and efficient server-side applications.

Traditionally javascript was used only for client-side web development inside browsers. With the introduction of node.js developers gained the ability to use javascript for backend development as well

History and evolution of node.js

Node.js was created in 2009 by ryan dahl before node.js web servers commonly used technologies such as php, java, and apache, servers, these technologies often handle multiple requests using separate threads, which consumed more memory and reduced performance under heavy traffic.

Ryan dahl developed node.js to solve the problem of handling many simultaneous connections efficiently he

built node.js on the powerful javascript engine called v8 which was originally developed for google chrome browser.

Important milestones

2009: node.js was officially introduced

2010: npm(node package manager) was introduced, making it easy to install libraries and packages

2011-2014: node.js become popular for real-time applications like chat applications and streaming services

2015: the node.js foundation was formed to support development

2019 onwards: node.js become widely used in enterprise applications apis, cloud computing, and microservices

Today node.js is used by major companies such as Netflix, paypal, linkedin, and uber

Architecture of node.js

Node.js uses an event-driven and non-blocking i/o architecture

1) Single threaded model

Unlike traditional servers that create a new thread for every request, node.js uses a single thread to handle multiple requests

This approach:

- Reduces memory usage

- Improves performance

- Handles thousands of connections efficiently

2) Event loop

The event loop is the core component of node.js

It continuously checks:

- Incoming requests

- Completed tasks

- Callback functions

The event loop allows node.js to perform asynchronous operations without blocking other tasks

3) Non-blocking i/o

Node.js performs operations like file reading, database queries and api requests asynchronously

Ex:

While one task is waiting for data
Node.js can continue handling other requests.
This makes node.js very fast and scalable

4) V8 javascript engine

Node.js uses the google v8 engine to convert
javascript code into machine code for faster
execution

High speed

Better performance

Efficient memory management

Key features of node.js

- Fast execution
- Asynchronous programming
- Scalability
- Cross platform
- Npm support
- Real time applications
- Open source
- Real time application
- Open source

Comparison: Node.js vs PHP vs Java

Node.js:

Language: javascript

Architecture: event-driven, non-blocking

Performance: very fast

Scalability: high

Best for: real time apps, APIs

Learning curve: easy for js developers

Concurrency: single thread with event loop

Package manager: npm

PHP:

Language: PHP

Architecture: blocking

Performance: moderate

Scalability: medium

Best for: dynamic websites

Learning curve: easy

Concurrency: thread/ request model

Package manager: composer

Java:

Language: java

Architecture: multi-threaded

Performance: fast

Scalability: high

Best for: enterprise applications

Learning curve: moderate to difficult

Concurrency: multi-thread

Package manager: maven/ gradle

Advantages of node.js over Traditional Technologies

Compared to PHP:

Faster for real-time applications

Better handling of multiple simultaneous users

Same language (JavaScript) for frontend and backend

Compared to java:

Simpler and lightweight

Requires less memory

Faster development

Node.js has transformed modern web development by allowing javascript to run on the server side. Its event-driven and non-blocking architecture makes it ideal for scalable and high-performance applications. Due to its speed , flexibility and large ecosystem, node.js has become one of the most popular backend technologies in the world.

Practical Assignment

Installing Node.js

Step 1: Download Node.js

Visit the official website:

[Node.js Official Website](https://nodejs.org/en/)

Download the **LTS (Long Term Support)** version.

Step 2: Install Node.js

Windows Installation

1. Open the downloaded .msi file
2. Click **Next**
3. Accept the license agreement
4. Choose installation location
5. Click **Install**
6. Finish the installation

7. Verify installation

Open command prompt and run:

Node -v

V22.0.0

Npm -v

Creating hello world application

Step 1: create a folder

Mkdir hello-node

Cd hello-node

Step 2: create a javascript file

Create a file named: app.js

Step 3: write the code

Console.log("hello world");

Step 4: run the application

Open terminal inside the project folder and run:

Node app.js

Output:

Hello world

Server.js:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.write("Hello World from Node.js");
  res.end();
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Node server.js

<http://localhost:3000>

Hello World from Node.js

Node.js is a powerful backend technology that uses JavaScript outside the browser. It is fast, scalable, lightweight, and suitable for modern web applications. By installing Node.js and creating a simple Hello World program, beginners can start learning server-side JavaScript development easily.

Node.js environment setup

Theory assignment

Role of npm in node.js development

Npm (node package manager) is the default package manager for node.js. it helps developers install, manage, and update libraries and packages required in node.js applications

Npm provides access to thousands of open-source packages that simplify development and reduce coding effort

Importance of npm

1. Package management : npm allows developers to install reusable libraries like express, react, lodash, and many others
2. Dependency management: it automatically manages project dependencies inside the node_modules folder and stores package information in package.json.
3. Script management: developer can define custom scripts such as start, test, and build inside package.json.

Example:

```
  "scripts": {  
    "start": "node app.js"  
  }
```

4. Version control: npm manages package versions to maintain compatibility between libraries

Common npm commands:

Npm init: initialize a new node.js project

Npm init -y: initialize project with default settings

Npm install package-name: install a package

Npm install: install all dependencies

Npm uninstall package-name: remove a package

Npm update: update installed packages

Npm start: run application

Npm run script-name: run custom script

Npm install -g nodemon: install package globally

Practical assignment

Practical assignment: json parsing with try...catch

```
function parseJSON(jsonString) {  
  try {  
    const data = JSON.parse(jsonString);  
    console.log("Parsed Data:", data);  
  } catch (error) {  
    console.log("Error parsing JSON:", error.message);  
  }  
}
```

// Valid JSON

```
parseJSON('{"name":"Devika","age":21}');
```

// Invalid JSON

```
parseJSON('{name:"Devika"}');
```

output:

Parsed Data: { name: 'Devika', age: 21 }

Error parsing JSON: Unexpected token n in JSON

Module system:

Theory assignment:

Javascript module system

A module is a reusable piece of code that can export variables, functions, or objects and import them into other files

Modules help organize code into smaller, manageable parts

Types of modules in javascript

Commonjs modules

Commonjs is the default module system used in node.js

`Module.exports = functionName;`

Import

`Const functionName = require("./file");`

ES modules(ESM)

Es modules use export and import

Export : `export default functionName;`

Import: `import functionName from "./file.js";`

Significance of Modules

1. Better Code Organization

Code can be divided into multiple files.

2. Reusability

Functions and classes can be reused in different files.

3. Maintainability

Large applications become easier to manage.

4. Avoid Global Scope Pollution

Variables remain inside their modules.

Practical Assignment

File 1: math.js

```
function add(a, b) {  
  return a + b;  
}
```

```
function subtract(a, b) {  
  return a - b;  
}
```

```
module.exports = {  
  add,
```

```
    subtract  
};
```

File 2: app.js

```
const math = require("./math");  
  
console.log("Addition:", math.add(10, 5));  
console.log("Subtraction:", math.subtract(10, 5));
```

Run the Program

```
node app.js
```

Output

```
Addition: 15  
Subtraction: 5
```

JavaScript APIs

Theory assignment

What are APIs?

Api stands for application programming interface. An api allows different software applications to communicate with each other

In web development, APIs are commonly used to:

Fetch data from servers

Send data to servers

Connect frontend applications with backend services

For example:

Weather apps use weather APIs

Social media apps use APIs for posts and users

Payment apps use payment APIs

JavaScript and APIs

JavaScript can interact with APIs using HTTP requests such as:

GET: fetch data

POST: send new data

PUT: update existing data

DELETE: remove data

JavaScript mainly uses the fetch for making network requests.

Fetch API

The fetch api is a modern JavaScript feature used to make asynchronous HTTP requests


```
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

Advantages of fetch API

1. Easy to use
Simple syntax for api requests
2. Promise-based
Supports .then() and .catch().
3. Asynchronous
Does not block the webpage while fetching data
4. Supports multiple HTTP methods
Get, post, put, delete

Practical assignment

Fetch data from json placeholder API

JSON Placeholder APL

This is a free fake API for testing and learning

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width, initial-scale=1.0">
  <title>Fetch API Example</title>

<style>
  body {
    font-family: Arial;
    padding: 20px;
  }

  input {
    padding: 8px;
    width: 200px;
  }

  button {
    padding: 8px 12px;
    cursor: pointer;
  }
```

```
.post {
    border: 1px solid #ccc;
    padding: 10px;
    margin-top: 10px;
    border-radius: 5px;
}

.error {
    color: red;
    font-weight: bold;
}
</style>
</head>
<body>

    <h1>JSONPlaceholder Posts</h1>

    <!-- Search Section -->
    <input type="number" id="userId"
placeholder="Enter User ID">
    <button
onclick="searchPosts()">Search</button>

    <br><br>

    <!-- Data Display -->
```

```
<div id="posts"></div>
```

```
<script>
```

```
    const postsDiv =  
    document.getElementById("posts");
```

```
    // Function to fetch all posts  
    function fetchPosts() {
```

```
        fetch("https://jsonplaceholder.typicode.com/posts  
")
```

```
            .then(response => response.json())
```

```
            .then(data => {
```

```
                postsDiv.innerHTML = "";
```

```
                data.forEach(post => {
```

```
                    const postElement =  
                    document.createElement("div");
```

```
                    postElement.classList.add("post");
```

```

        postElement.innerHTML = `
            <h3>${post.title}</h3>
            <p>${post.body}</p>
        `;

        postsDiv.appendChild(postElement);
    });
})

.catch(error => {

    postsDiv.innerHTML = `
        <p class="error">
            Failed to fetch data. Please try
again.
        </p>
    `;

    console.log("Error:", error);
});
}

// Function to search posts by user ID
function searchPosts() {

```

```
const userId =
document.getElementById("userId").value;

fetch(`https://jsonplaceholder.typicode.com/posts
?userId=${userId}`)

.then(response => response.json())

.then(data => {

    postsDiv.innerHTML = "";

    if (data.length === 0) {
        postsDiv.innerHTML = "<p>No posts
found.</p>";
        return;
    }

    data.forEach(post => {

        const postElement =
document.createElement("div");

        postElement.classList.add("post");
```

```

        postElement.innerHTML = `
            <h3>${post.title}</h3>
            <p>${post.body}</p>
        `;

        postsDiv.appendChild(postElement);
    });
})

.catch(error => {

    postsDiv.innerHTML = `
        <p class="error">
            Error fetching search results.
        </p>
    `;

    console.log("Error:", error);
});
}

// Load all posts when page opens
fetchPosts();

</script>

```

```
</body>  
</html>
```

Features implemented

Fetch API: fetches data from API

Display data: shows posts on webpage

Search functionality: search posts by user ID

Dynamic html: creates elements dynamically

Error handling: uses .catch() for errors

JSON parsing: converts response to JSON

How the program works

Fetch all posts

```
Fetchposts();
```

Convert response to JSON

```
Response.json()
```

Display data

```
Document.createElement("div");
```

Search posts

```
/posts?userId=1
```


Error Handling

`.catch(error =>{})`

Output

The webpage will:

- Show all posts initially
- Allow searching by User ID
- Display titles and bodies
- Show error messages if fetch fails

JavaScript APIs allow applications to communicate with servers and external services. Using the Fetch API, developers can retrieve, display, search, and manage data dynamically on web pages. Features like promises, `.then()`, and `.catch()` make asynchronous programming easier and more efficient in modern JavaScript development.

